

From a Bare Reach Template to Vision-Guided Pick-and-Place: Reinforcement Learning for a Kinova Gen3 Manipulator in Isaac Lab

Ido Raviel, Dan Butensky
Bar-Ilan University — AI Excellence Program
Guided by Prof. Gal Kaminka

September 1, 2026

Abstract

This report documents the development of a reinforcement-learning pick-and-place pipeline for a Kinova Gen3 arm with a Robotiq 2F-140 gripper, built in NVIDIA Isaac Lab and trained with PPO via `rs1_rl`. The project started from a generic, gripper-less Isaac Lab reach template and progressed through: (1) hardware integration of a parallel gripper, including an abandoned first attempt with a 2F-85 gripper before settling on the 2F-140; (2) an orientation-aware *reach* policy; (3) a *grasp* policy that was first trained from a single canonical start pose, found to fail when handed off from the reach policy in a chained rollout, and then retrained with IK-randomized start states for robustness; (4) a state-machine *chain* that composes the two frozen policies into a full reach $\rightarrow$ grasp $\rightarrow$ carry pick-and-place behavior; and (5) a vision-based extension that trains a convolutional pose-estimation network (**CubePoseNet**) from a single fixed workspace camera and substitutes its live predictions for the privileged, simulator-only cube pose used earlier in the pipeline. The report presents the exact reward structures, PPO hyperparameters, and training curves for each policy, several trial-and-error episodes that shaped the final design (a still-unresolved reach-wavering artifact and the IK-seeding strategy that unlocked stable grasp training), and the vision model’s measured accuracy (0.50 cm position error, 1.2° rotation error on a held-out test set), together with an honest account of what has *not* yet been measured — most notably, a persisted end-to-end success rate for the chain.

Contents

1	Introduction	2
2	Reach Policy	3
2.1	Task and algorithm	3
2.2	Reward structure	3
2.3	Randomization	3
3	Grasp Policy	4
3.1	Version (a): canonical-start baseline	4
3.2	Version (b): IK-randomized start states	4
3.2.1	IK mechanism	5

3.2.2	Reward retuning	5
3.3	Grasp PPO hyperparameters	6
4	Pick-and-Place Chain	7
4.1	State machine	7
4.2	Timing unification and action clipping	7
4.3	Visualization	8
5	Results Summary	8
5.1	Training curves	8
5.2	Qualitative results	8
6	Vision-Based Cube Pose Estimation	8
6.1	Motivation	8
6.2	Camera setup	8
6.3	Data collection	9
6.4	Dataset splitting	9
6.5	CubePoseNet: architecture	9
6.6	Training setup	10
6.7	Results	10
6.8	Live integration into the chain	10
7	Further Development	11
7.1	Handling the Reach-Wavering Artifact	11
7.2	Sim-to-Real Deployment	11
8	Conclusion	11

1 Introduction

The goal of this project is to train a Kinova Gen3 robot arm, fitted with a Robotiq 2F-140 parallel gripper, to autonomously pick a cube up off a table and carry it to a target pose in the air, using reinforcement learning trained entirely in simulation (NVIDIA Isaac Lab). Rather than training a single monolithic policy for the whole task, the project decomposes it into two specialized policies — *reach* and *grasp* — each trained independently with PPO, and then composes them at inference time into a single behavior via an explicit state machine. A later phase of the project replaces the privileged, simulator-only cube pose that both policies initially rely on with a pose estimate produced by a learned vision model observing a single fixed camera.

This repository extends [louislelay/kinova_isaac_lab_sim2real](#) with a full pick-and-place chain and a retrained robust grasp policy.

This report is organized to follow the project’s actual chronology: Section 2 covers the reach policy. Section 3 covers the grasp policy, including its most significant redesign. Section 4 covers the chain that combines both frozen policies into pick-and-place. Section 5 summarizes quantitative results across all of the above. Section 6 covers the vision-based pose estimation work. Section 7 covers further development (handling the reach-wavering artifact and sim-to-real deployment). Section 8 is a brief conclusion.

2 Reach Policy

2.1 Task and algorithm

The reach policy commands the arm’s seven joints so that the gripper’s fingertip frame (the `ee_frame`, offset 0.21 m from the wrist flange — not the flange itself) tracks a randomly sampled target position *and orientation*. Orientation tracking was added specifically so the policy could learn to approach the cube top-down, which the grasp policy requires for a reliable handoff.

The policy is trained with **PPO via `rsl_rl`**. Table 1 lists the exact hyperparameters, drawn from `gen3_reach/agents/rsl_rl_ppo_cfg.py` and the base `ReachEnvCfg` it inherits from.

Table 1: Reach policy: PPO hyperparameters.

Parameter	Value
Actor/critic network	MLP [64, 64], ELU activation
Initial action noise std	1.0
Number of parallel environments	4096 (env spacing 2.5 m)
Episode length	12.0 s
Steps per environment per update	24
Learning epochs per update	8
Mini-batches per update	4
Learning rate	1×10^{-3} , adaptive (target KL 0.01)
Discount factor γ	0.99
GAE λ	0.95
PPO clip parameter	0.2
Entropy coefficient	0.001
Value loss coefficient	1.0 (clipped value loss)
Max gradient norm	1.0
Max training iterations	1500

2.2 Reward structure

Table 2 lists every reward term defined in `gen3_reach/joint_pos_env_cfg.py`. Position and orientation tracking each use a coarse L2 term (always penalizing error) plus a fine-grained tanh-shaped bonus term (rewarding precise alignment near the target), a standard Isaac Lab pattern. Two smoothness terms are introduced via a *curriculum* that ramps their weight linearly from 0 at 25% of training to their final value at 60% of training, so that early exploration is not suppressed by a smoothness penalty before the policy has learned to reach at all.

2.3 Randomization

To make the reach policy robust to the states the grasp policy and the eventual chain would present it with, several quantities are randomized every episode:

- **Gripper state:** randomized open/closed at every reset, so the policy learns to reach correctly whether or not it is already holding an object.

Table 2: Reach policy: reward terms and weights.

Term	Weight	Description
<code>end_effector_position_tracking</code>	-0.6	coarse L2 position error
<code>end_effector_position_tracking_fine_grained</code>	$+0.6$	tanh bonus, std 0.07 m
<code>end_effector_orientation_tracking</code>	-0.3	coarse shortest-path angle
<code>end_effector_orientation_tracking_fine_grained</code>	$+0.30$	tanh bonus, std 0.5 rad
<code>action_rate</code>	$0 \rightarrow -0.005$ (curriculum)	action-smoothness penalty
<code>joint_vel</code>	$0 \rightarrow -0.0015$ (curriculum)	joint-velocity penalty

- **Payload mass:** an extra 0–0.3 kg is added to the gripper base link at random (the cube itself weighs 0.216 kg, comfortably inside this range), and the policy does not observe the mass directly — it must be robust to the load rather than compensate for a known value.
- **Target orientation range:** roll $\pm 10^\circ$; pitch 120° – 180° (with 180° , i.e. straight down, always included, since a top-down approach is required for grasping); yaw $\pm 45^\circ$ (covers all four faces of the cube).
- **Target position range:** height $z \in [0.05, 0.50]$ m.

3 Grasp Policy

The grasp policy takes over once the arm is positioned above the cube (as handed off by the reach policy, or by the chain’s state machine) and is responsible for lowering onto the cube, closing the gripper, and lifting it clear of the table. Unlike reach, grasp went through a substantial redesign driven directly by a failure discovered when the two policies were first chained together.

3.1 Version (a): canonical-start baseline

The first working version of grasp was trained from a single, near-fixed starting condition: the cube spawned at a fixed table position, $[0.43, 0, 0.055]$, with only a small ± 1 cm reset jitter in x and y (none in z), and the arm always started from the same canonical joint configuration directly above it. Its reward structure is summarized in Table 3 (leftmost column), and it trained successfully in isolation: given exactly the start condition it was trained on, the policy reliably grasped and lifted the cube.

The problem: in a chained rollout, the reach policy does not hand off from one single canonical joint configuration — it arrives above the cube from a wide variety of valid arm configurations (whatever the orientation- and payload-randomized reach policy happens to produce). Grasp version (a), having only ever seen one starting condition, was not robust to this variation and failed when actually chained after reach.

3.2 Version (b): IK-randomized start states

The fix was to retrain grasp so that every episode resets to a random, physically valid start state: a random cube pose on the table, with the arm’s joints IK-placed so the gripper sits directly top-down above the cube. This is implemented in `gen3_grasp/mdp/ik_reset.py` (function `reset_above_cube_ik`).

3.2.1 IK mechanism

Inverse kinematics (IK) answers a simple question: given a desired position and orientation for the gripper, what joint angles make the arm reach it? We used a numerical IK solver to precompute, for many different randomly sampled cube poses on the table, a matching valid arm configuration with the gripper placed directly top-down above the cube. This let us build a large bank of realistic, varied starting states for training the grasp policy, instead of relying on a single hand-picked canonical start pose.

Two refinements to this basic idea mattered in practice:

- A **decaying null-space pull toward the canonical pose** is added during the solve: $k_{\text{null}} = 0.6 \times (1 - \text{iter}/120)$, i.e. it starts at 0.6 and decays linearly to 0 across the 120 iterations. This keeps the (kinematically redundant) arm’s null-space motion biased toward a natural, canonical-like configuration early in the solve, while still letting the position/orientation task term converge tightly by the end.
- Seeds themselves are initialized near the canonical joint configuration $[0, 0.3, 0, 1.8, 0, 0.7, 0]$ plus uniform noise (0.5 rad), rather than fully random — and among the seeds that converge, the solver picks the one with **minimum L2 distance to the canonical configuration**.
- To keep training fast, IK is not solved live at every reset. It is run once at startup to build a table of $\sim 10,000$ precomputed valid start states (built in chunks of 2500 to bound GPU memory, cached per device), and each subsequent episode reset simply samples a row from this table.

Why the null-space pull and closest-to-canonical selection mattered. An early version of the IK reset, without this seeding and selection strategy, produced **contorted** start poses — valid IK solutions, but kinematically unnatural ones. Training from these caused the arm to perform a visible “circle/unwind” motion at the start of every episode as it worked its way out of the contorted configuration, and learning stalled under this regime. Seeding IK near canonical, decaying the null-space pull rather than holding it fixed, and explicitly selecting the seed closest to canonical among converged solutions together produced natural, elbow-up start poses and was the single change that most improved training progress — learning proceeded noticeably faster than under the contorted-pose regime.

Training-speed note. Solving IK live at every reset was reported at the time to run at roughly 7 seconds per training iteration, which is far too slow for RL training; switching to the precomputed table restored training throughput to approximately 58,000 environment steps per second (full simulation speed). These two throughput figures are carried over from earlier project notes rather than independently re-benchmarked while preparing this report, and should be read as recalled order-of-magnitude figures rather than freshly measured ones.

An additional **cube jitter** of ± 4 cm was applied laterally to the cube’s placement (on top of the randomized start state), specifically so the policy has to localize the cube rather than assume it sits directly beneath the gripper.

3.2.2 Reward retuning

The reward structure was retuned as the robust-grasp retrain progressed; Table 3 shows the before-and-after.

Table 3: Grasp policy: reward weights before and after the robustness retrain.

Term	Version (a)	Current
<code>lifting_object</code>	disabled	+1.0 (min. height 0.05 m)
<code>ee_to_cube_l2</code> (coarse)	—	−1.0
<code>ee_to_cube</code> (tanh, std 0.1)	−0.2	−0.2
<code>cube_to_target</code> (L2)	−1.0	−1.0
<code>cube_to_target_fine</code> (tanh)	+0.25	+1.0
Place-target height range	[0.30, 0.65] m	[0.08, 0.20] m
<code>action_rate</code> (curriculum)	-5×10^{-4}	-1×10^{-3}
Max training iterations	1000	1300

The most significant single change is **re-enabling `lifting_object`** (a hard, non-shaped +1 reward for every step the cube’s height exceeds a small threshold), which was disabled in version (a) and had previously been removed altogether in an even earlier iteration (`de7e790`, “Remove `object_lifted_binary` reward from grasp task”). Re-adding it was specifically motivated by a **local optimum** the policy fell into without it: hovering or orbiting near the cube, collecting the shaped distance rewards, without ever actually committing to grasp and lift. Alongside it, a new coarse L2 term for end-effector-to-cube distance (`ee_to_cube_l2`) was added to strengthen the pull toward the cube before the fine tanh term takes over at close range, and the place-target height range was shrunk from a tall [0.30, 0.65] m column down to a much shallower [0.08, 0.20] m band, making the “carry it somewhere reasonable” part of the task substantially easier so training could focus on the grasp itself.

3.3 Grasp PPO hyperparameters

Grasp uses a noticeably different PPO configuration from reach (Table 4), reflecting the fact that grasping is a higher-dimensional, contact-rich task: a deeper network, more conservative entropy and learning-rate settings, and empirical observation/action normalization (neither of which reach uses).

Table 4: Grasp policy: PPO hyperparameters (differences from reach in bold).

Parameter	Value
Actor/critic network	MLP [128, 64, 64] (3 layers, vs. reach’s [64, 64])
Steps per environment per update	30 (vs. 24)
Learning epochs per update	5 (vs. 8)
Entropy coefficient	0.005 (vs. 0.001)
Learning rate	1×10^{-4} (vs. 1×10^{-3} , i.e. 10× lower)
Discount factor γ	0.98 (vs. 0.99)
Action clipping	± 1.0 (reach: none)
Empirical normalization	enabled (reach: disabled)
Max training iterations	1300

4 Pick-and-Place Chain

With reach and grasp each frozen and trained independently, the two policies are composed at inference time into a single pick-and-place behavior via an explicit **state machine**, implemented in `scripts/rs1_rl/play_lift_and_place.py` against the `Gen3-LiftAndPlace-Chain-v0` environment. The chain drives the gripper top-down above the cube using the reach policy, hands off to the grasp policy to pick the cube up, and hands back to the reach policy to carry the cube to a randomly sampled target pose in the air.

4.1 State machine

The chain runs five phases: `REACH` $\rightarrow$ `PAUSE1` $\rightarrow$ `GRASP` $\rightarrow$ `PAUSE2` $\rightarrow$ `CARRY`. (The project’s own higher-level description collapses the two brief pauses into their neighboring phases and refers to this informally as “three phases”: reach, grasp, and reach/carry.)



- **Reach $\rightarrow$ Pause1 (hybrid handoff — whichever fires first):**
 - close and settled: distance to target < 0.05 m *and* end-effector speed < 0.03 m/s; or
 - plateaued: no further progress for 30 steps (≈ 1 s) *and* distance < 0.07 m; or
 - a hard timeout of 120 steps (≈ 4 s), so the chain always proceeds rather than stalling indefinitely on a poor approach.
- **Pause1 $\rightarrow$ Grasp:** a fixed 0.3 s hold (reach policy keeps holding position) before handing control to grasp.
- **Grasp $\rightarrow$ Pause2:** triggers when the cube’s height is within 2 cm of a 0.155 m lift target. There is no timeout on this transition — a grasp attempt that has not yet lifted the cube keeps trying until the episode itself ends.
- **Pause2 $\rightarrow$ Carry:** after a fixed hold, the gripper command is frozen at grasp’s last (closed) value, and a random carry target is sampled.
- **Carry:** the reach policy drives to a randomly sampled top-down pose in the air ($x \in [0.40, 0.60]$, $y \in [-0.25, 0.25]$, $z \in [0.25, 0.40]$, pitch kept within reach’s own trained orientation range to avoid out-of-distribution behavior).

4.2 Timing unification and action clipping

Because the chain runs two independently trained policies inside one simulated environment, their simulation timing had to be made identical: both the grasp environment and the chain environment use $dt = 1/60$ s with a control decimation of 2, matching the timing the reach policy itself was trained under. Each policy’s own trained action-clipping bound (read from its agent configuration, e.g. grasp’s ± 1.0) is applied to that policy’s raw output immediately after inference, and the two clipped action tensors are merged by the current phase using `torch.where` before being sent to the simulator.

4.3 Visualization

The chain overlays colored target markers so the active goal is visible during playback (e.g. the current reach target and the carry target), and the fully tuned version of the chain script and environment shortened the episode length from 10.0s to 5.0s and the post-grasp pause from 0.3s to 0.1s so that episodes don’t idle at the carry target for the remainder of their length, alongside a marker-color and default-debug-sphere cleanup for cleaner recordings.

This tuning and the vision work described in Section 6 were originally developed on separate branches that had diverged from each other; both have since been reconciled into a single history, so the vision-based chain now uses the same tuned timing and marker values described above.

5 Results Summary

5.1 Training curves

Both the reach and grasp policies were trained multiple times as the reward structure and randomization were iterated on; the runs referenced below are the final, currently deployed checkpoints.

Reach. Across training, the fine-grained position-tracking reward rises from near zero (≈ 0.0003) to ≈ 0.33 – 0.36 ; the fine-grained orientation-tracking reward rises to ≈ 0.16 – 0.19 ; the mean episode reward rises from roughly -0.2 to $+4.0$ – 4.5 . Essentially all episodes run to the time-out termination (fraction ≈ 1.0), i.e. reach has no meaningful early-failure termination mode.

Grasp. Across the robustness-retrain runs, the `lifting_object` reward (mean per-step “cube is lifted” signal, max 1.0 if lifted at every step) rises from ≈ 0 to 0.87 – 0.88 ; the fine-grained `cube_to_target` reward rises to 0.21 – 0.57 depending on the run; the object-dropping termination fraction stays low, 0.0016 – 0.0064 , i.e. dropping the cube after picking it up is rare by the end of training; mean episode reward rises to 3.97 – 6.37 .

5.2 Qualitative results

The full chain — reach, grasp, and carry — **works end-to-end**, and a screen-recorded demo of a full pick-and-place rollout is committed to the repository at `medias/kinova_video.mp4` and embedded in the project README. “Works end-to-end” is, at this stage, a qualitative, video-verified claim rather than a measured success percentage over N episodes.

6 Vision-Based Cube Pose Estimation

6.1 Motivation

Every policy described so far conditions on the cube’s pose as computed from **privileged simulator state** (in earlier stages, via ground-truth object pose and IK; described in the project’s working notes as “ground truth used to set the reach/grasp targets”). The vision phase of the project replaces this privileged signal with a pose estimate produced by a neural network observing a single fixed RGB camera, so that the pipeline no longer depends on simulator-only information.

6.2 Camera setup

A fixed `CameraCfg` was added to the chain environment (`gen3_lift_and_place/joint_pos_env_cfg.py`): a pinhole camera at world position $(1.319, 0, 1.153)$, oriented to look down at approximately $(0.45, 0, 0.055)$ with roughly a 50° downward tilt, focal length 24 mm, 20.955 mm horizontal aperture,

clipping range $[0.1, 4.0]$ m, capturing 640×480 RGB frames. The placement was deliberately chosen to sit beyond the cube’s reachable x -range and centered in y , so that the descending gripper does not fully occlude the cube from view during the grasp phase. A further implementation detail worth noting: because the robot’s base sits at the world origin with identity rotation, the camera’s world-frame pose *is* already its extrinsics in the robot-root frame that both policies are trained in, avoiding an extra frame-conversion step downstream.

6.3 Data collection

`scripts/rsl_rl/collect_vision_data.py` runs the same frozen reach $\rightarrow$ pause1 $\rightarrow$ grasp $\rightarrow$ pause2 $\rightarrow$ carry state machine used for playback, across 32 parallel environments by default, and saves a frame and label at (almost) every step instead of only rendering. The two near-static pause phases are subsampled 1-in-5 to avoid wasting the dataset on near-duplicate frames, while REACH/GRASP/CARRY frames are saved every step; a “settled” latch stops saving further CARRY frames for an environment once its cube has arrived and stopped moving, to the same end. Each collected frame is saved as an RGB PNG, and a corresponding entry is appended to `labels.jsonl` recording the environment id, simulation step, current phase, the cube’s position and orientation (quaternion) in the robot-root frame, and eight projected 2D cube-corner pixel coordinates (a hand-written pinhole projection, kept for a possible future keypoint/PnP-based approach but not used by the model actually trained). A single `meta.json` records image resolution, the camera’s intrinsic matrix, and its extrinsics in the robot-root frame.

The dataset actually collected for training comprises **12,000 frames** with the following phase distribution: 6494 CARRY, 3320 REACH, 2065 GRASP, 64 PAUSE1, 57 PAUSE2.

6.4 Dataset splitting

`scripts/rsl_rl/split_vision_dataset.py` performs a fixed-count random split, not an exact phase-stratified one: records are shuffled with a fixed seed (42) and then sliced into 1000 test / 1000 validation / 10,000 train frames. Shuffling before slicing (rather than splitting along rollout order, which would front-load REACH frames) means each split ends up with a broadly representative mix of phases in practice, and the script prints a per-split phase count as a sanity check, but it does not explicitly enforce matching phase proportions across splits.

6.5 CubePoseNet: architecture

CubePoseNet (defined in `source/gen3/gen3/tasks/manager_based/gen3_lift_and_place/mdp/vision_obs.py`) uses an ImageNet-pretrained **ResNet-18** backbone with its final classification layer replaced by an identity mapping, feeding a small MLP head ($512 \rightarrow 256 \rightarrow 9$, ReLU) on top. The 9-dimensional output is split into a 3D position and a **6D continuous rotation representation** [1], decoded into a full 3×3 rotation matrix via Gram-Schmidt orthogonalization. This representation was chosen explicitly over quaternions to avoid the antipodal sign ambiguity that quaternions introduce for regression losses. Input frames are resized to 240×320 , ImageNet-normalized, and (at training time only) augmented with color jitter (brightness, contrast, saturation ± 0.2 , hue ± 0.02).

The network regresses **full 3-DoF orientation**, not merely yaw, on the grounds that once the cube is gripped it tilts freely — the training script’s own documentation notes that 84% of CARRY-phase frames are not yaw-only — and that the cube’s distinctly colored faces make full orientation observable from a single image in the first place.

6.6 Training setup

`scripts/rsl_rl/train_cube_pose.py` trains `CubePoseNet` with AdamW (learning rate 3×10^{-4} , weight decay 1×10^{-4}), a cosine annealing schedule over 30 epochs, batch size 64, and mixed-precision training. The loss is

$$\mathcal{L} = \text{MSE}(\hat{p}, p) + 0.1 \cdot \text{MSE}(\hat{R}, R),$$

i.e. mean-squared error on position plus a matrix-MSE term on the rotation matrix (weighted $0.1\times$) — a simple Frobenius-style penalty at training time, though rotation *evaluation* is reported in degrees via a proper geodesic error metric. Both the last and best (lowest validation position error) checkpoints are saved each epoch; only the best checkpoint, `pretrained_models/cube_pose/best.pt` (45 MB), is kept in the repository.

6.7 Results

On the held-out, never-trained-on 1000-frame test split, `CubePoseNet` achieves:

0.50 cm position error, 1.2° rotation error

Accuracy is not uniform across the task: it is best while the cube sits unoccluded on the table before the grasp ($\approx 0.3\text{--}0.4\text{ cm}$), and worst immediately after the cube is lifted, when the closed gripper partially occludes it from the camera’s view ($\approx 1\text{ cm}$). Even the worst case remains small relative to the cube’s own size ($\approx 7.2\text{ cm}$). These numbers come from `eval_cube_pose.py`, which loads a checkpoint and reports per-phase position and rotation error on a chosen split.

6.8 Live integration into the chain

This is not merely an offline-evaluated model: it is wired directly into a live chain rollout. A new environment, `Gen3-LiftAndPlace-Chain-Vision-v0`, subclasses the existing chain environment and **overrides only the grasp policy’s two cube-pose observation terms** (`object_position` and `object_orientation`) with vision-based equivalents that call `CubePoseNet` on the live camera image — reusing the exact same slot in the frozen grasp policy’s observation vector, so the policy itself is entirely unmodified; only the *source* of that observation changes. The model is loaded once and cached on the environment, and its prediction is computed once per simulation step and shared by every observation term and by the play script, rather than re-run redundantly.

`play_lift_and_place.py` gained a `-vision` flag that switches to this environment and replaces the ground-truth cube pose used for (a) the reach target during the initial top-down approach, (b) the in-progress lift-height check during grasp, and (c) the grasp→pause2 transition condition, with the vision model’s live prediction. The reach target is specifically *re-sampled from the latest prediction on every reach step*, not only once at episode reset, as an explicit fix for one-step camera lag immediately following a reset. Ground truth is retained only for a diagnostic prediction-error printout, not for any control decision.

The vision-enabled play script does compute and print a running (episodes-carried / episodes-completed) success fraction to the console as episodes reset, but this is a live console readout, not a value that has been captured and saved anywhere in the repository.

7 Further Development

7.1 Handling the Reach-Wavering Artifact

The reach policy currently exhibits a small, persistent oscillation once it settles at its target — a learned wavering behavior present even under deterministic evaluation, not exploration noise. Several reward-shaping strategies (weak and strong joint-velocity penalties, both immediate and curriculum-ramped, as well as a proximity-gated penalty) were tried and each ran into the same tradeoff: any penalty strong enough to suppress the wavering also suppressed the reach itself, since a velocity-based penalty cannot distinguish approaching the target from oscillating at it. The most promising direction not yet attempted is at the **physics level** rather than the reward level: the arm’s wrist/elbow actuators are comparatively underdamped (damping 4/2 against stiffness 60/25), and raising actuator damping should suppress the oscillation directly without penalizing the reach itself. This would require retraining reach under the new damping and re-validating the resulting control gains before any hardware deployment.

7.2 Sim-to-Real Deployment

The project has been developed and validated entirely in simulation. Moving the pick-and-place chain onto the physical Kinova Gen3 would need to address, at minimum: matching the real arm’s controller gains to the simulated stiffness/damping values so trained behavior transfers (particularly if the damping change above is made); calibrating the real camera’s intrinsics and extrinsics for **CubePoseNet**, which was trained only on rendered simulation images and would need to be validated against, or fine-tuned on, real camera images; and re-checking the Robotiq 2F-140’s real actuation timing and grip-force behavior, since its simulated model is only an approximation of the physical gripper’s dynamics. None of this has been attempted yet.

8 Conclusion

The project has produced a working, end-to-end simulated pick-and-place pipeline on a Kinova Gen3 + Robotiq 2F-140 platform, composed from two independently trained PPO policies, and has extended that pipeline with a vision-based cube-pose estimator that is already measured (0.50 cm / 1.2° test-set error) and already wired into a live **-vision** chain mode. The items in Section 7 are the natural next steps for the project.

References

- [1] Y. Zhou, C. Barnes, J. Lu, J. Yang, H. Li. *On the Continuity of Rotation Representations in Neural Networks*. CVPR 2019.